Pebbles, Metropolis, needles

Dominique Chu

School of Computing
University of Kent, UK
D.F.Chu@kent.ac.uk

Natural Computation, University of Kent

Pebbles on the beach

We assume that the length of the square is $2a$, such that the radius of the circle is a.

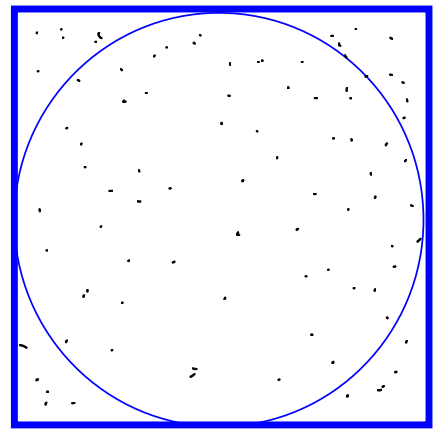
$$\frac{\text{number of pebbles in circle}}{\text{number of pebbles in square}} =$$

$$= \frac{\text{Area of circle}}{\text{Area of square}} =$$

$$= \frac{\pi\left(a^2\right)}{\text{Area of square}} =$$

$$= \frac{\pi\left(a^2\right)}{4a^2} =$$

$$= \frac{\pi}{4}$$

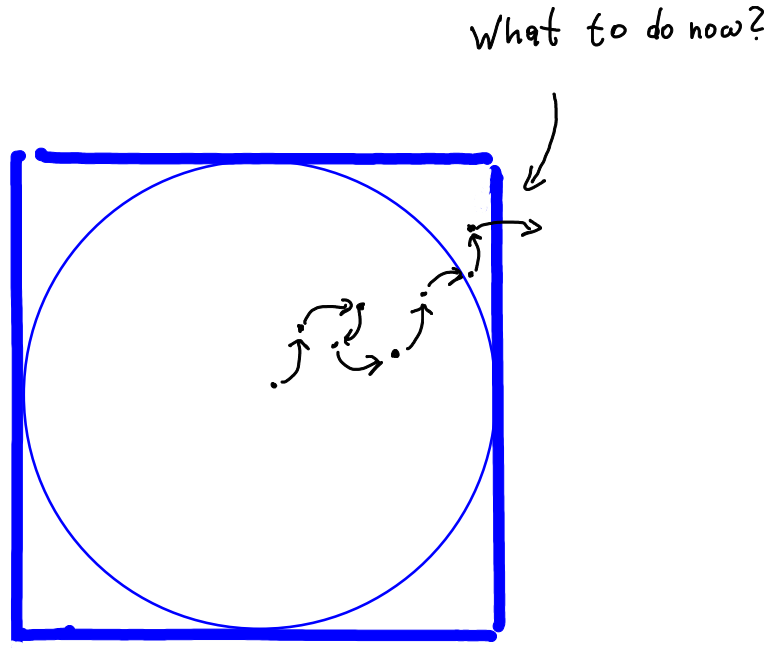


Pebbles on the beach

Imagine now, that your square is too large to be able to throw into it, and you need to walk through it...
So, instead, you:

- Take a bag full of pebbles and go to the square.
- You throw a pebble into a random direction and a random distance (within some bounds).
- You then move to the location of this pebble and throw again.
- You repeat this untill there are many pebbles in the square, then you count the pebbles in the square and circle, as above.

Question: What to do if you throw a pebble outside of the square?



There are two possibilities

1. Ignore and continue throwing until you land inside
2. Continue throwing from outside (and hope you will get back).

Both are wrong!

procedure `markov-pi`

$N_{\text{hits}} \leftarrow 0;\ \{x, y\} \leftarrow \{1, 1\}$ (initialise)

for $i = 1, \ldots, N$ **do**

$\delta x \leftarrow \texttt{ran}\,(-\delta, \delta)$ } choose where to throw next pebble.

$\delta y \leftarrow \texttt{ran}\,(-\delta, \delta)$

if $(|x + \delta x| < 1$ and $|y + \delta y| < 1)$ **then** ← is new position inside of square?

$x \leftarrow x + \delta x$

$y \leftarrow y + \delta y$

if $(x^2 + y^2 < 1)\ N_{\text{hits}} \leftarrow N_{\text{hits}} + 1$ ← are we still inside circle?

output N_{hits}

Discussion Questions

- If you interpret this as a MC, what are the states of the MC?
- What does steady state mean in this case?
- What is the probability to find at least 1 pebble at an (arbitrarily chosen) position (x, y) in steady state?

Detailed balance condition

The detailed balance condition **must** be obeyed in equilibrium! It is very important, so please pay attention now:

$$\pi_1 p(1 \to 2) = \pi_2 p(2 \to 1)$$
$$\pi_1 p(1 \to 3) = \pi_3 p(3 \to 1)$$
$$\pi_1 p(1 \to 4) = \pi_4 p(4 \to 1)$$
$$\pi_2 p(2 \to 3) = \pi_3 p(3 \to 2)$$
$$\dots$$

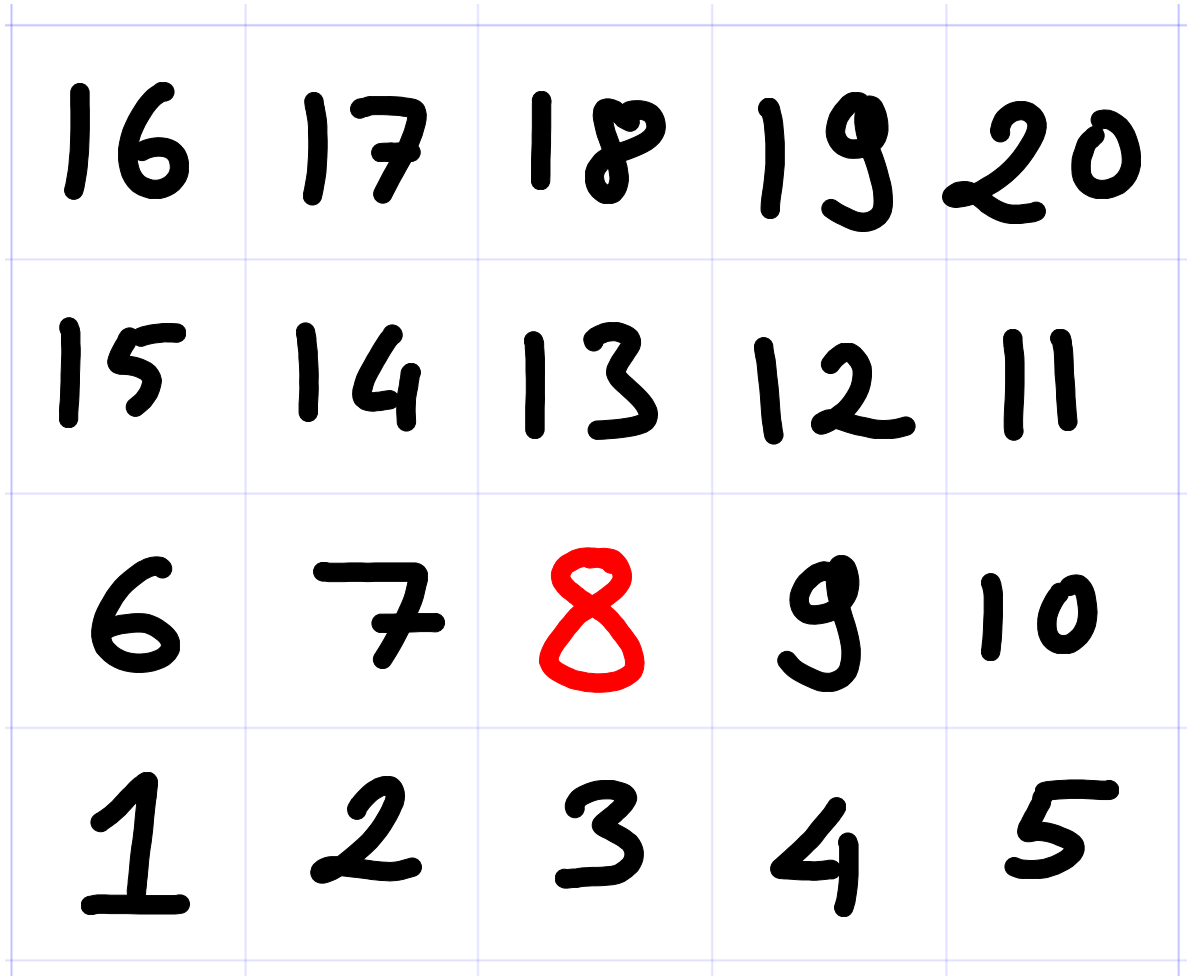
Here, π_i is the steady state probability to find the system in state i, and $p(i \to j)$ is the probability that the system transitions from state i to state j.

Metropolis algorithm

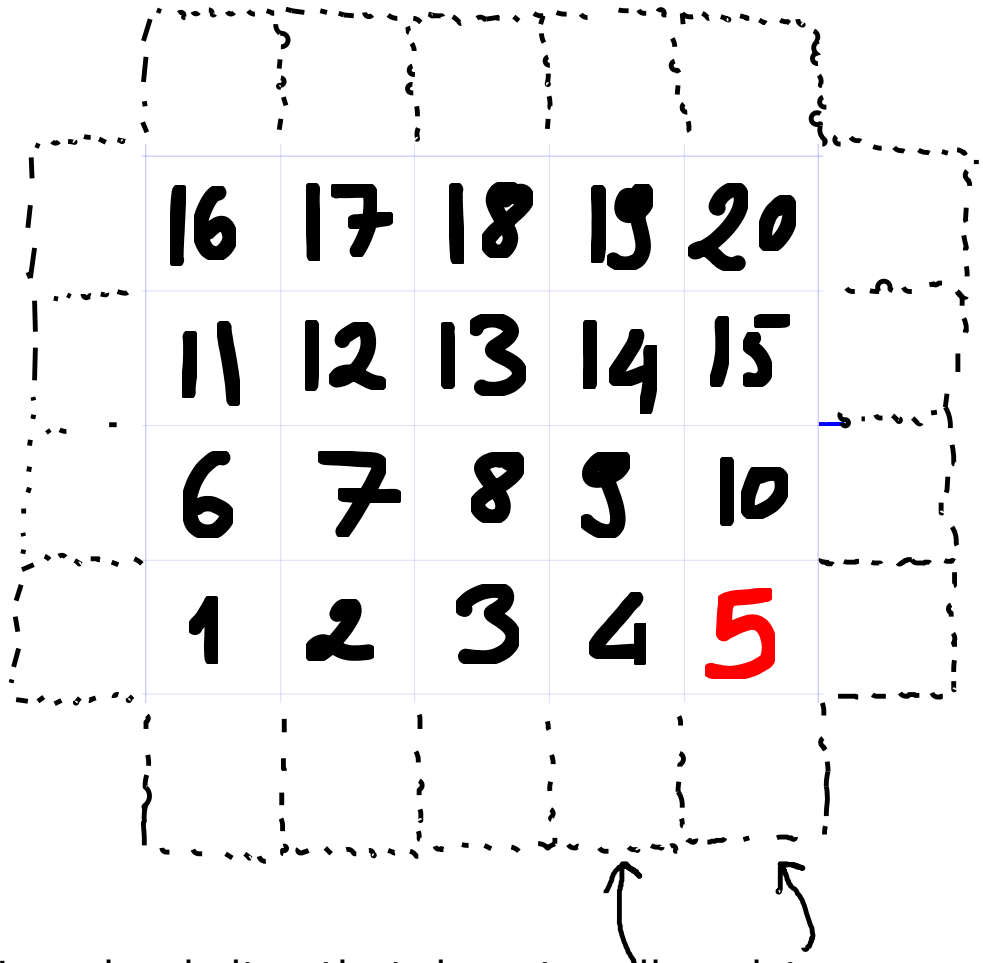
We have a Markov chain M with states $1, 2, \ldots, n$ and we want that the steady state probability to be in state i is given by a particular value π_i (which we choose). The question is now: How do we set the transition rates/probabilities between states so that we can achieve this desired steady state probability?

1. Start at a random state `a`.
2. Current state is `a`. Propose a new random state `b`.
3. Accept the proposed move to new state `b` with the following acceptance probability
$$P_{\text{acc}} = \min\left(1, \frac{\pi(b)}{\pi(a)}\right)$$
4. Goto 2.

- Assume a turtle can go north/south/east/west
- How do I set the probabilities so that **in the long run** the probabilities to be at any site are equal?

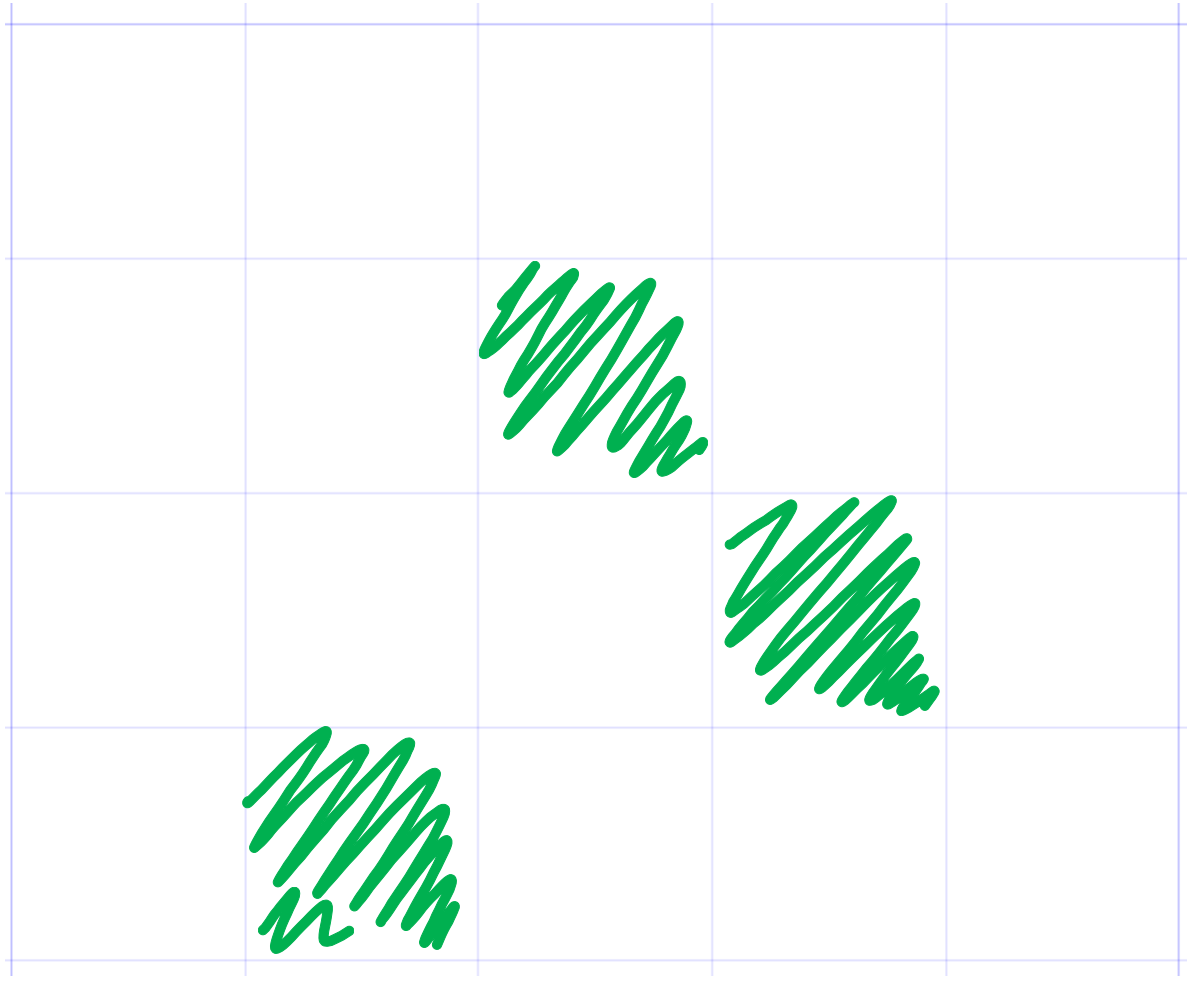


What if the turtle is in the corner?



Imagined sites that do not really exist,
i.e. they have an probability of 0 to be occupied

A more complicated case



Imagine now, that we want $\pi(\text{green}) > \pi(\text{white})$. How can we achieve that?

Using the Metropolis algorithm

1. Take initially a random state of the Markov chain.
2. Call the current state `a`. Propose a new state `b`.
3. Calculate the acceptance probability

$$P_{\text{acc}} = \min\left(1, \frac{\pi(b)}{\pi(a)}\right)$$

4. Draw a random number (between 0 and 1). Let us call this number `r`.
5. If $\texttt{r} < P_{\text{acc}}$ then new state is `b`, otherwise remain in current state.
6. `GOTO 2`.

If you move through the Markov chain in this way, then in the long-run you will be in state i with probability $\pi(i)$.

Be careful with these distinctions

- Probability for the system/turtle to be in a particular state.
- Steady state probability.
- Transition probability.
- Transition rate.
- Acceptance probability.

Using the Metropolis algorithm to calculate transition probabilities for the turtle

We can use the Metropolis algorithm in order to simulate moving through a Markov chain (according to the algorithm on the last page), but we can also use it to calculate fixed transition probabilities that give us desired long-term probablities. To do this do the following:

1. Calculate the acceptance probabilties for each pair of states.
2. Calculate the probability that you propose a given state (this will be a fixed value in most cases).
3. Multiply these values. This will be the transition probability.
4. If the transition probabilites to leave a state do not add to 1, then also add a self-transition so that all probabilties add to 1.

Next question: Finding steady state probabilities

Assume now that we have a discrete time MC and are given the transition probabilities. How can we find the steady state probabilities?

1. Construct a Markov chain with states $1, 2, 3, \ldots N$ ("the state labels").
2. Initialise an array `count[]` of size $N + 1$ and initialise all elements to 0.
3. Start in a random initial state of your Markov chain.
4. Do n times (outer loop):
   - 4.1 Do m times (inner loop):
     - 4.1.1 Perform a `TOWER SAMPLING` to decide next state.
     - 4.1.2 Transition into new state.
     - 4.1.3 Set `i` to the current state label.
   - 4.2 `count[i] += 1`
5. Calculate the estimated probability for π_i for state `i` as:

$$\pi_i \approx \frac{\texttt{count[i]}}{\texttt{n}}$$

Here `n` and `m` must be very large numbers.

1. Let `k` be the number of possible transitions from current state (including self-transitions).
2. We associate the transitions with numbers in order of probability, such that the first transition is the one with highest probability and the k-th transition is the lowest probability. Equal probabilities are ordered randomly.
3. Let `t[]` be an array of size $k + 1$. Now set:
   - 3.1 Set `t[0]` to 0.
   - 3.2 Set `t[1]` to the probability of transition 1.
   - 3.3 Set `t[2]` to the probability of transition 2.
   - 3.4 ...
   - 3.5 Set `t[k]` to the probability of transition `k`.
4. Draw a random number r between 0 and 1.
5. Find the index `l` such that

$$\sum_{i=0}^{l-1} \texttt{t[i]} < r \leq \sum_{i=0}^{l} \texttt{t[i]}$$

TOWER SAMPLING: Example

Asume that the current state is state 4 and there are 3 possible transitions, namely:

- To state 4 with probability 0.1
- To state 1 with probability 0.7
- To state 94 with probability 0.2

This gives us the following values:

- We draw the random number $r = 0.97365234$.
- `t= [ 0,0.7,0.2,0.1]`
- We calculate:
  - `t[0]= 0`
  - `t[0] + t[1]= 0.7`
  - `t[0] + t[1] + t[2]= 0.9`
  - `t[0] + t[1] + t[2] + t[3]= 1`
- Hence we choose 3 transition to state 4

TOWER SAMPLING: Example

Asume that the current state is state 4 and there are 3 possible transitions, namely:

- To state 4 with probability 0.1
- To state 1 with probability 0.7
- To state 94 with probability 0.2

This gives us the following values:

- We draw the random number $r = 0.73652341$.
- `t= [ 0,0.7,0.2,0.1]`
- We calculate:
  - `t[0]= 0`
  - `t[0] + t[1]= 0.7`
  - `t[0] + t[1] + t[2]= 0.9`
  - `t[0] + t[1] + t[2] + t[3]= 1`
- Hence we choose transition 2 to state 94

TOWER SAMPLING: Example

Asume that the current state is state 4 and there are 3 possible transitions, namely:

- To state 4 with probability 0.1
- To state 1 with probability 0.7
- To state 94 with probability 0.2

This gives us the following values:

- We draw the random number $r = 0.173652341$.
- `t= [ 0,0.7,0.2,0.1]`
- We calculate:
  - `t[0]= 0`
  - `t[0] + t[1]= 0.7`
  - `t[0] + t[1] + t[2]= 0.9`
  - `t[0] + t[1] + t[2] + t[3]= 1`
- Hence we choose transition 1 to state 1.

Simulating the a CTMC

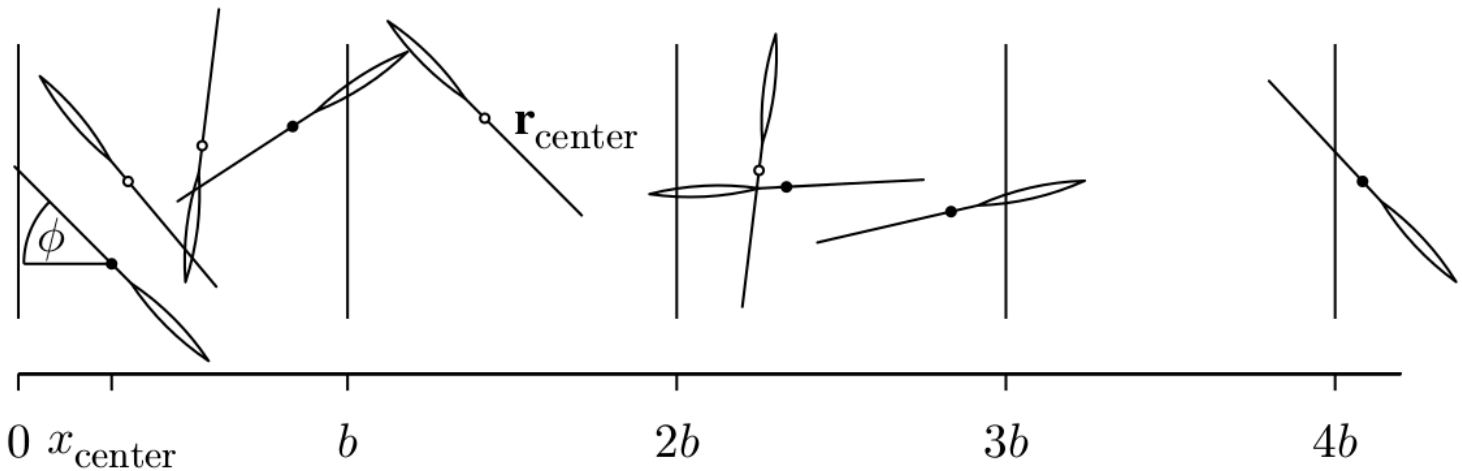
This is almost the same as a DTMC, but you also need to calculate the waiting time in current state.

1. Choose a random initial state and set time to $T = 0$.
2. Call your current state state `a`. Assume that there are I state transitions from `a` to other states. This means you are given the transition rates $k_1, k_2, \ldots, k_I$. Define now
$$P = \sum_{j=1}^{I} k_j$$
3. Draw a random number $0 < r < 1$.
4. Calculate the waiting time Δt as follows:
$$\Delta t = -\frac{1}{P} \ln(r)$$
5. Update time: $T = T + \Delta t$.
6. Calculate the transition probabilities for each state and choose the new state as you would for the DTMC.
7. `GOTO 2`.

Bonus Material below

Needles on the floor

- Imagine throwing needles randomly onto the floor.
- a . . . length of the needle.
- b . . . distance between cracks in the floor.
- ϕ . . . angle relative to cracks.
- x_{centre}.

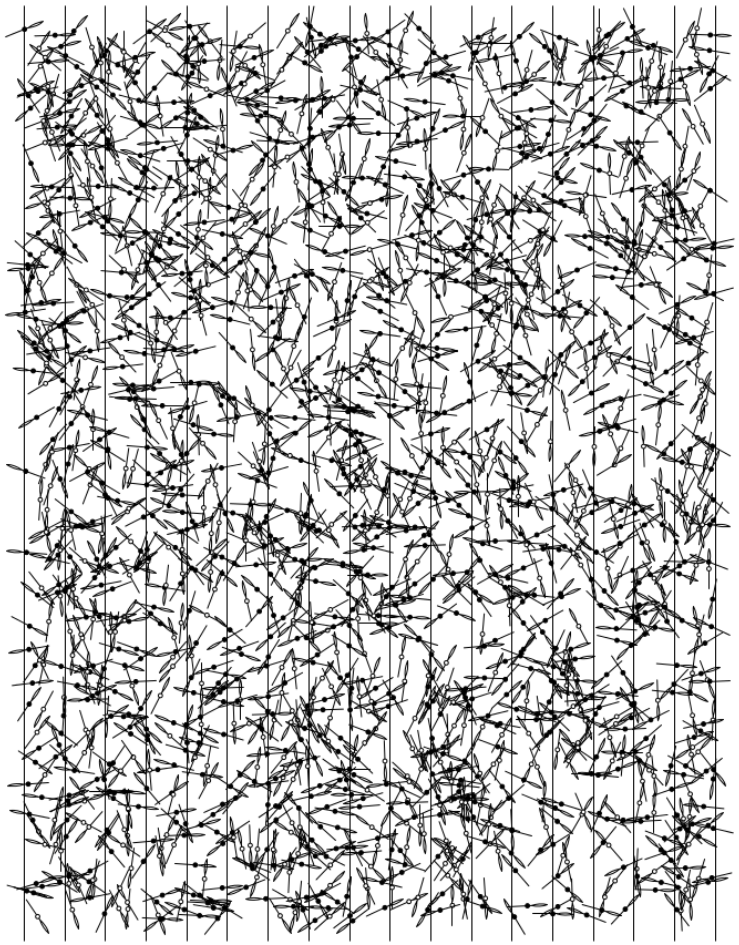


How many needles hit a crack?

$$N_{\text{hit}}(x, \phi) =$$

$$= \begin{cases} 1 & x < a/2 \& |\phi| < \arccos(2x/a) \\ 0 & \text{otherwise} \end{cases}$$

This, however, contains π implicitly.



An algorithm to calculate π

procedure `direct-needle(patch)`

$x_{\text{center}} \leftarrow \texttt{ran}\,(0, b/2)$

1 $\delta x \leftarrow \texttt{ran}\,(0, 1)$

$\delta y \leftarrow \texttt{ran}\,(0, 1)$

$|\mathbf{x}| \leftarrow \sqrt{\delta x^2 + \delta y^2}$

if $(|\mathbf{x}| > 1)$ **goto** 1

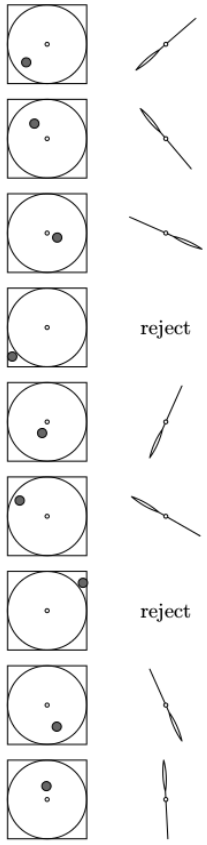
$x_{\text{tip}} \leftarrow x_{\text{center}} - \frac{1}{2}a \cdot \delta x/|\mathbf{x}|$ | see next slide

$N_{\text{hits}} \leftarrow 0$

if $(x_{\text{tip}} < 0)$ $N_{\text{hits}} \leftarrow 1$

output N_{hits}

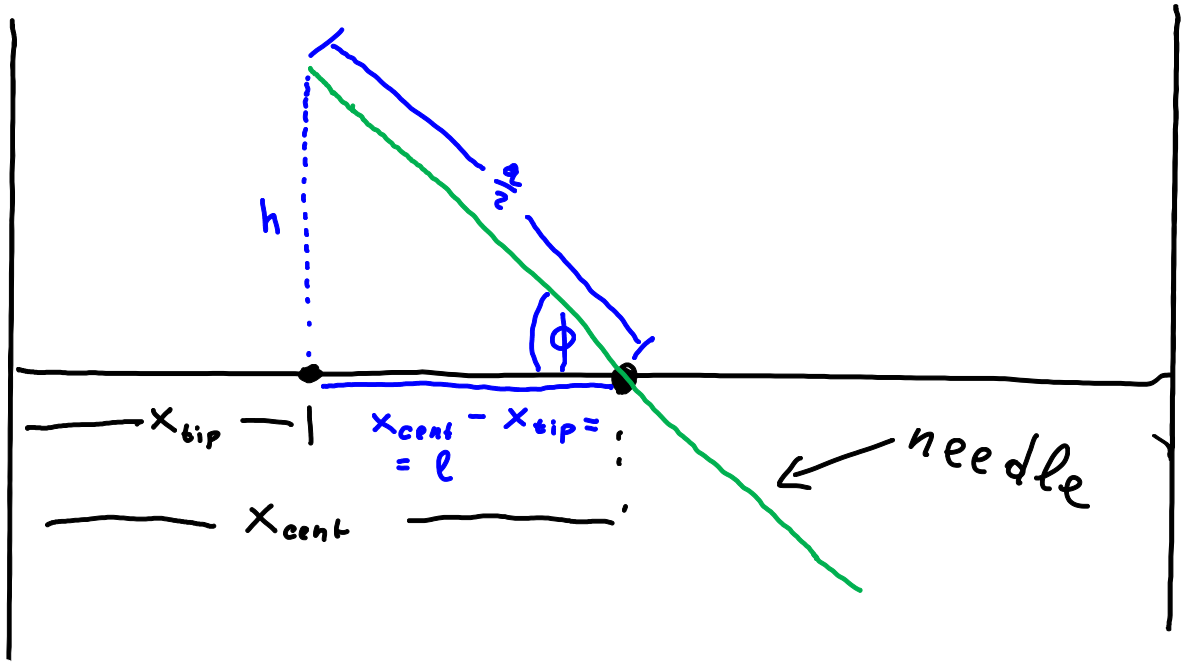
Position of tip



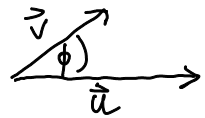
$$x_{\text{tip}} = x_{\text{centre}} - (x_{\text{centre}} - x_{\text{tip}}) = x_{\text{centre}} - l$$

$$\frac{\sin\left(\frac{\pi}{2}\right)}{\frac{a}{2}} = \frac{\sin\left(\frac{\pi}{2} - \phi\right)}{l}$$

$$l = \frac{a}{2}\cos(\phi)$$

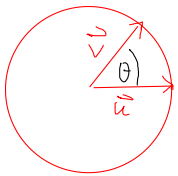


2.) $\cos(\phi) = \frac{\vec{u}\cdot\vec{v}}{|\vec{u}|\cdot|\vec{v}|} = \frac{\delta x}{|x|} \Leftrightarrow \boxed{l = \frac{a}{2}\cdot\frac{\delta x}{|x|}}$



$\vec{u} = \begin{pmatrix} \delta x \\ \delta y \end{pmatrix} \quad \vec{v} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \Rightarrow \vec{u}\cdot\vec{v} = \delta x$

$|\vec{u}| = |x| \qquad |\vec{v}| = 1$



$\vec{v} = \begin{pmatrix} \delta x \\ \delta y \end{pmatrix} \quad \vec{u} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

$\Rightarrow \vec{u}\cdot\vec{v} = \delta x \cdot 1 + \delta y \cdot 0 = \underline{\delta x}$

$\cos\theta = \left|\frac{\vec{u}\cdot\vec{v}}{|\vec{u}|\cdot|\vec{v}|}\right|$

$\Rightarrow \cos\theta = \frac{\delta x}{|\vec{u}|\cdot|\vec{v}|}$

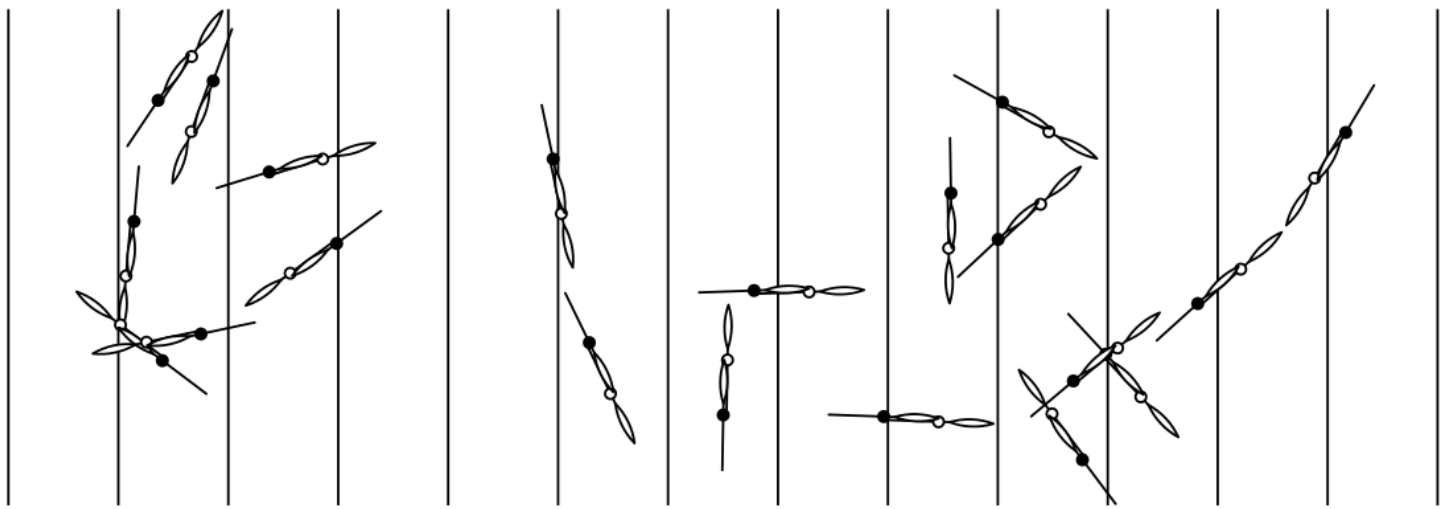
$|\vec{u}| = 1 \qquad |\vec{v}| = |x|$

1. Simulate using the above algorithm.
2. Think (no need to simulate)

- Imagine composite needles (2 glued together) are thrown.
- We cannot know which one of the two needles was thrown (symmetry).
- Any point of the composite needles is equally likely to hit a crack.
- We conclude therefore that the number of hits on average is proportional to the length of the needles, i.e.

$$N_{\text{hit}} = K \cdot \text{length of needle} \tag{1}$$

We can also show that N_{hit} does not depend on the shape of the needle.



What is K

- The question is now: What is the proportionality constant K?
- Consider a round needle of radius $r = \frac{b}{2}$.
- The length of the needle is then the circumference of the circle, which is:

$$a = 2\pi\frac{b}{2} = \pi b$$

- We known that the number average number of hits in this case is exactly 2, i.e.

$$N_{\text{hit}} = 2 = K \cdot \text{length of needle} = K \cdot \pi b$$

We thus obtain the value of K:

$$K = \frac{2}{\pi b}$$

Thus, given any needle length, we can now calculate π as follows, just by counting the number of hits of needles from equation 1:

$$\pi = \frac{2}{bN_{\text{hit}}} \cdot \text{length of needle}$$